BSc. (Hons.) Engineering in Information and Communication Engineering (ICE)

Conclusion and Future Work

Engineering Research (Embedded Systems) Project

Student Name: J.M.Widanegamage

Index No: 21UG0623

Student Name: K.P.G.H.D.Dissanayake

Index No: 21UG0618

Date of Submission: 1/12/2023

Alarm Clock Development using PIC Microcontroller and RTC

Key findings and contributions:

We did prepare our project using breadboard. We identified some problems have it. Some of them are, Breadboard had some errors, can't handle old push buttons.
Finally we solved our problems and breadboard simulations correctly worked. Then we moved dotboard simulations.

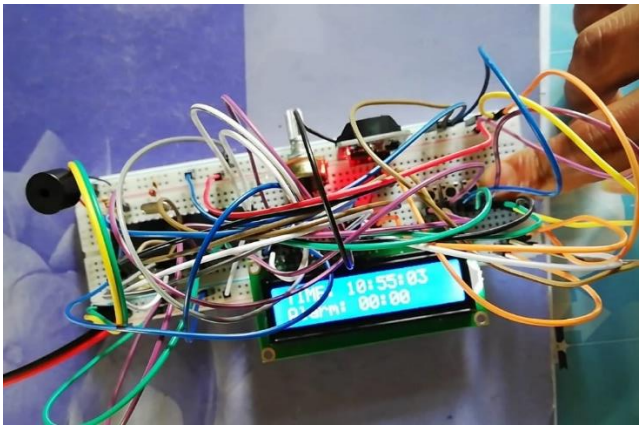
Limitations:

Alarm clocks typically jolt us awake with a jarring sound or vibration, disrupting our natural sleep cycle. This sudden transition from deep sleep to wakefulness can cause grogginess, stress, and even confusion. Some alarm clocks emit bright light, which can suppress melatonin production and make it harder to fall back asleep if you wake up before your natural wake time. By understanding the limitations of alarm clocks and exploring alternative wake-up methods, we can prioritize better sleep health and wake up feeling more refreshed and energized.

Potential Areas for Future Research or Improvements:

You can set the alarm according to our existing system. When the buzzer is on, you can turn it off. Further, when we upgrade the system, we intend to connect it to a phone and switch it on/off.
We intended to create it to determine whether they had done the work we needed to see whether we were not in that position at that moment or not.

Conclusion and Relevance:



This is our working alarm clock. Still we can set the alarm and turn it off. Here we decided to use a Wi-Fi module to send a message to the phone after the alarm is turned off. To ensure that the higher officials have started the work and to evaluate that the employees who have to do the work have time for the

```c
/*
 * Program: Alarm Clock Using PIC
 */
#pragma config FOSC = HS
#pragma config WDTE = OFF
#pragma config PWRTE = ON
#pragma config BOREN = ON
#pragma config LVP = OFF
#pragma config CPD = OFF
#pragma config WRT = OFF
#pragma config CP = OFF
#define _XTAL_FREQ 20000000
//Define the LCD pins
#define RS RD2
#define EN RD3
#define D4 RD4
#define D5 RD5
#define D6 RD6
#define D7 RD7
//Define Buttons
#define MB RB1
#define LB RB0
#define RB RB2
#define UB RB3
#define BB RB4
//Define Buzz
#define BUZZ RD1 //Buzzer is connected to RD1
/*Set the current value of date and time below*/
int sec = 00;
int min = 55;
int hour = 10;
int date = 06;
int month = 05;
int year = 18;
/*Vars for Alarm clock*/
char set_alarm = 0;
char trigger_alarm = 0;
char pos = 8;
char jump = 0;
char alarm_h0 = 0;
char alarm_h1 = 0;
char alarm_m0 = 0;
char alarm_m1 = 0;
int alarm_val[4] = {0, 0, 0, 0};
#include <xc.h>
#include "lcd.h"
#include "PIC16F877a_DS3231.h"
```

```
int main() {
    TRISD = 0x00; //Make Port D pins as outptu for LCD interfacing
    TRISB = 0xFF; //Switchs are declared as input pins
    OPTION_REG = 0b00000000; //Enable pull up Resistor on port B for switchs
    BUZZ = 0; //Turn of buzzer
    Lcd_Start(); // Initialize LCD module
  //Give an intro message on the LCD
         Lcd_Clear();
         Lcd_Set_Cursor(1,1);
         Lcd_Print_String("Alarm Clock");
         Lcd_Set_Cursor(2,1);
         Lcd_Print_String(" SLTC ");
         __delay_ms(1500);

    while (1) {
        Update_Current_Date_Time(); //Read the current date and time from RTC module
        //Split the into char to display on lcd
        char sec_0 = sec % 10;
        char sec_1 = (sec / 10);
        char min_0 = min % 10;
        char min_1 = min / 10;
        char hour_0 = hour % 10;
        char hour_1 = hour / 10;
        //Display the Current Time on the LCD screen
        Lcd_Clear();
        Lcd_Set_Cursor(1, 1);
        Lcd_Print_String("TIME: ");
        Lcd_Print_Char(hour_1 + '0');
        Lcd_Print_Char(hour_0 + '0');
        Lcd_Print_Char(':');
        Lcd_Print_Char(min_1 + '0');
        Lcd_Print_Char(min_0 + '0');
        Lcd_Print_Char(':');
        Lcd_Print_Char(sec_1 + '0');
        Lcd_Print_Char(sec_0 + '0');
// the Display Date on the LCD screen

        Lcd_Set_Cursor(2, 1);

        Lcd_Print_String("Alarm: ");

        Lcd_Print_Char(alarm_val[0] + '0');
        Lcd_Print_Char(alarm_val[1] + '0');
        Lcd_Print_Char(':');
        Lcd_Print_Char(alarm_val[2] + '0');
        Lcd_Print_Char(alarm_val[3] + '0');

        __delay_ms(50);
```

```
        //Use middle button to check if alarm has to be set
        if (MB == 0 && set_alarm == 0) { //If middle button is pressed and alarm is not turned on
            while (!MB); //Wait till button is released
            set_alarm = 1; //start setting alarm value
        }
        if (MB == 0 && set_alarm == 1) { //If middle button is pressed and alarm is not turned off
            while (!MB); //Wait till button is released
            set_alarm = 0; //stop setting alarm value
        }
        //If alarm has to be navigate through each digit
        if (set_alarm == 1) {
            if (LB == 0) { //If left button is pressed
                while (!LB); //Wait till button is released
                pos--; //Then move the cursor to left
            }
            if (RB == 0) { //If right button is pressed
                while (!RB); //Wait till button is released
                pos++; //Move cursor to right
            }

            if (pos >= 10) //eliminate ":" symbol if cursor reaches there
            {
                jump = 1; //Jump over the ":" symbol
            } else
                jump = 0; //get back to normal movement


            if (UB == 0) { //If upper button is pressed
                while (!UB); //Wait till button is released
                alarm_val[(pos - 8)]++; //Increase that particular char value
            }
            if (BB == 0) { //If lower button is pressed
                while (!UB); //Wait till button is released
                alarm_val[(pos - 8)]--; //Decrease that particular char value
            }

            Lcd_Set_Cursor(2, pos + jump);
            Lcd_Print_Char(95); //Display "_" to indicate cursor position
        }

        //IF alarm is set Check if the set value is equal to current value
        if (set_alarm == 0 && alarm_val[0] == hour_1 && alarm_val[1] == hour_0 && alarm_val[2] == min_1 && alarm_val[3] ==
min_0)
            trigger_alarm = 1; //Turn on trigger if value match


        if (trigger_alarm) { //If alarm is triggered
            //Beep the buzzer
```

```
            BUZZ = 1;
            __delay_ms(500);
            BUZZ = 0;
            __delay_ms(500);
        }

        __delay_ms(200);//Update interval

    }
    return 0;
}
```